

中国海洋大学

《系统开发工具基础》实验报告

授课教师：周小伟

实验名称	第一次实验 (Shell 基础与文件系统 / Git / LaTeX)
姓名	桂佳锋
学号	25020007030
班级	25 计算机科学与技术
实验日期	2026 年 8 月 24 日
代码仓库	https://github.com/GJF080117/jf (见第 4 节)

表 1: 实验基本信息

1 实验检查（课上给助教检查的内容和结果）

本节记录课上完成并提交给助教检查的 4 道题目的题目要求、作答过程、运行结果。

1.1 第 1 题：含空格文件名的批量整理 (Shell 基础与文件系统)

题目描述

在当前目录新建 q01，并完全使用命令行完成下列任务：

1. 创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）；
2. 显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件；
3. 把 input 下所有.txt 文件复制到 work/< 学号 >/，保留相对目录结构并正确处理名称中的空格；
4. 将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

作答过程与关键代码

```
1 mkdir -p q01/input/docs q01/input/tmp
2 printf 'alpha\nbeta\n' > 'q01/input/docs/notes one.txt'
3 printf 'hidden' > q01/input/docs/.secret.txt
4 touch q01/input/tmp/empty.txt
5 printf 'log line 1\nlog line 2\n' > q01/input/run.log
```

Listing 1: 创建目录与文件（含空格、隐藏文件）

```
1 readlink -f q01 # 输出 q01 的绝对路径
2 ls -la q01/input # -a 显示隐藏文件，-l 长格式
```

Listing 2: 显示绝对路径、长格式列出（含隐藏文件）

```
1 mkdir -p "q01/work/25020007030"
2 find q01/input -name '*.txt' -exec sh -c '
3     for f; do
4         dir="q01/work/25020007030/${f#q01/input/}"; dir="${dir%/*}"
5         mkdir -p "$dir"; cp "$f" "$dir/"
6     done' _ {} +
```

Listing 3: 复制.txt 文件到 work/< 学号 >/ 并保留相对目录结构

```
1 chmod 750 q01/work/25020007030
2 chmod 640 q01/work/25020007030/input/docs/*.txt q01/work
   /25020007030/input/tmp/*.txt
```

```

3 find q01/work/25020007030 -type f | sed 's|q01/work/25020007030/||'
\
4 | while read f; do echo "$f $(wc -c < "q01/work/25020007030/$f"
)"; done \
5 | sort > q01/inventory.txt

```

Listing 4: 设置权限并生成 inventory.txt

```

jf@ubuntu: ~/q01
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

jf@ubuntu:~$ mkdir q01
jf@ubuntu:~$ cd q01
jf@ubuntu:~/q01$ mkdir -p input/docs input/tmp
jf@ubuntu:~/q01$ cat > "input/docs/notes one.txt" <<EOF
> alpha
> beta
> EOF
jf@ubuntu:~/q01$ echo "hidden" > input/docs/.secret.txt
jf@ubuntu:~/q01$ touch input/tmp/empty.txt
jf@ubuntu:~/q01$ cat > input/run.log <<EOF
> 2026-08-24 start task
> 2026-08-24 finish prepare
> EOF
jf@ubuntu:~/q01$ pwd
/home/jf/q01
jf@ubuntu:~/q01$ ls -la input
total 20
drwxrwxr-x 4 jf jf 4096 Aug 24 00:15 .
drwxrwxr-x 3 jf jf 4096 Aug 24 00:09 ..
drwxrwxr-x 2 jf jf 4096 Aug 24 00:12 docs
-rw-rw-r-- 1 jf jf 48 Aug 24 00:15 run.log
drwxrwxr-x 2 jf jf 4096 Aug 24 00:13 tmp
jf@ubuntu:~/q01$ mkdir -p work/25020007030
jf@ubuntu:~/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007030/ \;
find: 'cp--parents': No such file or directory
find: 'cp--parents': No such file or directory
find: 'cp--parents': No such file or directory
jf@ubuntu:~/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007030/ \;
jf@ubuntu:~/q01$ mkdir -p work/25020007030
jf@ubuntu:~/q01$ find work/25020007030 -type d -exec chmod 750 {} \;
jf@ubuntu:~/q01$ find work/25020007030 -type f -exec chmod 640 {} \;
jf@ubuntu:~/q01$ find work/25020007030 -type f -printf "%P %s\n" > inventory.txt

```

图 1: 第 1 题完整命令行作答过程

运行结果

```

1 input/docs/.secret.txt 7
2 input/docs/notes one.txt 11
3 input/tmp/empty.txt 0

```

Listing 5: inventory.txt 内容 (相对路径 + 字节数)

其中 notes one.txt 含文件名中的空格，在复制和统计时通过引号正确进行了处理；.secret.txt 为隐藏文件，ls -a 才能显示。inventory.txt 中给出了各文件的相对路径和实际字节数。

```

jf@ubuntu: ~/q01
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

jf@ubuntu:~$ mkdir q01
jf@ubuntu:~$ cd q01
jf@ubuntu:~/q01$ mkdir -p input/docs input/tmp
jf@ubuntu:~/q01$ cat > "input/docs/notes one.txt" <<EOF
> alpha
> beta
> EOF
jf@ubuntu:~/q01$ echo "hidden" > input/docs/.secret.txt
jf@ubuntu:~/q01$ touch input/tmp/empty.txt
jf@ubuntu:~/q01$ cat > input/run.log <<EOF
> 2026-08-24 start task
> 2026-08-24 finish prepare
> EOF
jf@ubuntu:~/q01$ pwd
/home/jf/q01
jf@ubuntu:~/q01$ ls -la input
total 20
drwxrwxr-x 4 jf jf 4096 Aug 24 00:15 .
drwxrwxr-x 3 jf jf 4096 Aug 24 00:09 ..
drwxrwxr-x 2 jf jf 4096 Aug 24 00:12 docs
-rw-rw-r-- 1 jf jf 48 Aug 24 00:15 run.log
drwxrwxr-x 2 jf jf 4096 Aug 24 00:13 tmp
jf@ubuntu:~/q01$ mkdir -p work/25020007030
jf@ubuntu:~/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007030/ \;
find: 'cp--parents': No such file or directory
find: 'cp--parents': No such file or directory
find: 'cp--parents': No such file or directory
jf@ubuntu:~/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007030/ \;
jf@ubuntu:~/q01$ mkdir -p work/25020007030
jf@ubuntu:~/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007030/ \;
jf@ubuntu:~/q01$ find work/25020007030 -type d -exec chmod 750 {} \;
jf@ubuntu:~/q01$ find work/25020007030 -type f -exec chmod 640 {} \;
jf@ubuntu:~/q01$ find work/25020007030 -type f -printf "%P %s\n" > inventory.txt

```

图 2: 第 1 题命令行操作与运行结果

1.2 第 2 题: 随机访问日志统计 (Shell 管道与文本处理)

题目描述

在 q02 中创建 access.csv, 并使用给定数据完成日志统计:

1. 编写 analyze.sh, 接收一个 CSV 文件路径; 文件不存在时将错误写入 stderr 并返回非零退出码;
2. 使用 Shell 管道以及 awk、sort、head 等工具, 输出 HTTP 5xx 数量最多的前 2 个 path; 按次数降序, 次数相同时按 path 字典序;
3. 输出全部数据行的平均 latency_ms, 保留两位小数; 表头不得计入;
4. 分别对 access.csv 和一个不存在的文件运行脚本; 不得使用 Python、电子表格或手工统计。

作答过程与关键代码

```

1 #!/bin/bash
2 csv_file="$1"
3 if [ ! -f "${csv_file}" ]; then
4     echo "error: file ${csv_file} does not exist" >&2
5     exit 1
6 fi
7 # 统计 5xx 最多的前 2 个 path: 按次数降序、次数相同按字典序
8 awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}' "${csv_file}" \

```

```

9 | sort | uniq -c | sort -k1,1nr -k2,2 | head -n 2
10 # 平均 latency_ms, 保留两位小数 (跳过表头)
11 awk -F',' 'NR>1 {sum += $5; cnt++;} END {print sum, cnt}' "${
    csv_file}" \
12 | awk '{printf "%.2f\n", $1/$2}'

```

Listing 6: analyze.sh 完整脚本

```

jff@ubuntu:~/q02
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

jff@ubuntu:~/q02$ mkdir q02
jff@ubuntu:~/q02$ cd q02
jff@ubuntu:~/q02$ cat > access.csv <<EOF
> timestamp,user,path,status,latency_ms
> 09:00:01,alice,/api/users,200,120
> 09:00:02,bob,/api/orders,500,310
> 09:00:03,alice,/api/users,503,180
> 09:00:04,carol,/login,500,90
> 09:00:05,bob,/api/orders,502,260
> 09:00:06,dave,/health,200,20
> 09:00:07,alice,/api/orders,500,420
> 09:00:08,carol,/api/users,500,150
> EOF
jff@ubuntu:~/q02$ cat > analyze.sh <<'EOF'
> #!/bin/bash
> csv_file="$1"
> if [ ! -f "$csv_file" ]; then
>   echo "error: file $csv_file does not exist" >&2
>   exit 1
> fi
> awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}' "$csv_file" \
> | sort \
> | uniq -c \
> | sort -k1,1nr -k2,2 \
> | head -n 2
> awk -F',' 'NR>1 {sum += $5; cnt++;} END {print sum, cnt}' "${
> | awk '{printf "%.2f\n", $1/$2}'
> EOF
jff@ubuntu:~/q02$ chmod +x analyze.sh
jff@ubuntu:~/q02$ ./analyze.sh access.csv
    3 /api/orders
    2 /api/users
193.75
jff@ubuntu:~/q02$ ./analyze.sh noexist.csv
error: file noexist.csv does not exist
jff@ubuntu:~/q02$

```

图 3: 第 2 题 access.csv 与 analyze.sh 的作答过程

运行结果

对 access.csv 运行:

```

1 $ bash analyze.sh access.csv
2     3 /api/orders
3     2 /api/users
4 193.75

```

Listing 7: 对 access.csv 运行脚本

对不存在的文件运行:

```

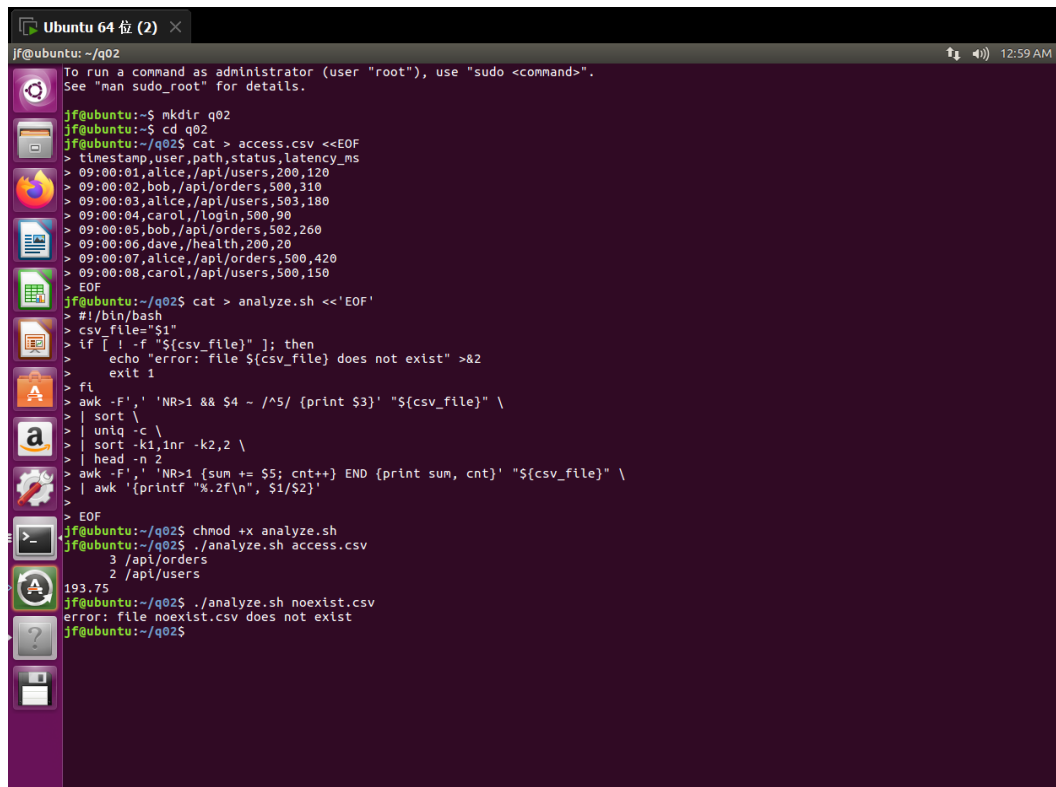
1 $ bash analyze.sh not_exist.csv
2 error: file not_exist.csv does not exist
3 $ echo $?

```

4 1

Listing 8: 对不存在文件运行脚本

对不存在文件的错误信息写入了 stderr (>&2), 并通过 `exit 1` 返回非零退出码; 5xx 状态行中 `/api/orders` 出现 3 次、`/api/users` 出现 2 次, 为前 2 名; 平均延迟 $(120 + 310 + 180 + 90 + 260 + 20 + 420 + 150)/8 = 193.75$ 。



```
Ubuntu 64 位 (2) x
jf@ubuntu: ~/q02
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

jf@ubuntu:~/q02$ mkdir q02
jf@ubuntu:~/q02$ cd q02
jf@ubuntu:~/q02$ cat > access.csv <<EOF
> timestamp,user,path,status,latency_ms
> 09:00:01,alice,/api/users,200,120
> 09:00:02,bob,/api/orders,500,310
> 09:00:03,alice,/api/users,503,180
> 09:00:04,carol,/login,500,90
> 09:00:05,bob,/api/orders,502,260
> 09:00:06,dave,/health,200,20
> 09:00:07,alice,/api/orders,500,420
> 09:00:08,carol,/api/users,500,150
> EOF
jf@ubuntu:~/q02$ cat > analyze.sh <<'EOF'
> #!/bin/bash
> csv_file="$1"
> if [ ! -f "$csv_file" ]; then
>   echo "error: file $csv_file does not exist" >&2
>   exit 1
> fi
> awk -F',' 'NR>1 && $4 ~ /^5/ {print $3}' "$csv_file" \
> | sort \
> | uniq -c \
> | sort -k1,1nr -k2,2 \
> | head -n 2
> awk -F',' 'NR>1 {sum += $5; cnt++} END {print sum, cnt}' "$csv_file" \
> | awk '{printf "%.2f\n", $1/$2}'
> EOF
jf@ubuntu:~/q02$ chmod +x analyze.sh
jf@ubuntu:~/q02$ ./analyze.sh access.csv
3 /api/orders
2 /api/users
193.75
jf@ubuntu:~/q02$ ./analyze.sh noexist.csv
error: file noexist.csv does not exist
jf@ubuntu:~/q02$
```

图 4: 第 2 题 access.csv、analyze.sh 及运行结果

1.3 第 3 题：制造、解决并解释一次合并冲突 (Git)

题目描述

在 q03 中从零创建一个 Git 仓库，并主动制造、解决一次合并冲突：

1. 初始化仓库，在 main 创建 config.txt，内容为 mode=normal，并完成初始提交；
2. 创建 feature-a，把内容改为 mode=safe 并提交；从初始 main 创建 feature-b，把内容改为 mode=fast 并提交；
3. 回到 main，先合并 feature-a，再合并 feature-b；解决冲突时保留 mode=safe，并另加一行 note=reviewed；
4. 确认工作区干净，运行 git log --all --graph --decorate --oneline 查看历史。

作答过程与关键代码

```
1 cd q03 && git init
2 printf 'mode=normal\n' > config.txt
3 git add config.txt && git commit -m "init: mode=normal"
4 git switch -c feature-a
5 printf 'mode=safe\n' > config.txt
6 git add config.txt && git commit -m "feature-a: set mode=safe"
7 git switch master
8 git switch -c feature-b
9 printf 'mode=fast\n' > config.txt
10 git add config.txt && git commit -m "feature-b: set mode=fast"
```

Listing 9: 初始化并创建分支提交

```
1 git switch master
2 git merge feature-a           # 第一次合并：无冲突
3 git merge feature-b           # 第二次合并：产生冲突（同一行被不同修
    改）
4 # 编辑 config.txt，保留 mode=safe，另加一行 note=reviewed
5 git add config.txt
6 git commit -m "merge feature-b: resolve conflict, keep safe + note"
7 git status                   # 确认工作区干净
8 git log --all --graph --decorate --oneline
```

Listing 10: 依次合并并解决冲突

```
Setting up git-man (1:2.7.4-0ubuntu1.10) ...
Setting up git (1:2.7.4-0ubuntu1.10) ...
jf@ubuntu:~/q03$ git init
Initialized empty Git repository in /home/jf/q03/.git/
jf@ubuntu:~/q03$ git config user.name "jf"
jf@ubuntu:~/q03$ git config user.email "jf@example.com"
jf@ubuntu:~/q03$ echo "mode=normal" > config.txt
jf@ubuntu:~/q03$ git add config.txt
jf@ubuntu:~/q03$ git commit -m "init: mode=normal"
[master (root-commit) d3a0388] init: mode=normal
1 file changed, 1 insertion(+)
create mode 100644 config.txt
jf@ubuntu:~/q03$ git checkout -b feature-a
Switched to a new branch 'feature-a'
jf@ubuntu:~/q03$ echo "mode=safe" > config.txt
jf@ubuntu:~/q03$ git add config.txt
jf@ubuntu:~/q03$ git commit -m "feature-a: set mode=safe"
[feature-a 5a08307] feature-a: set mode=safe
1 file changed, 1 insertion(+), 1 deletion(-)
jf@ubuntu:~/q03$ git checkout main
error: pathspec 'main' did not match any file(s) known to git.
jf@ubuntu:~/q03$
```

图 5: 第 3 题初始化仓库、创建分支的作答过程

运行结果

解决冲突后的 config.txt:

```
1 mode=safe
2 note=reviewed
```

Listing 11: config.txt 最终内容

git log --all --graph --decorate --oneline 输出:

```
1 * 25c1246 (HEAD -> master) merge feature-b: resolve conflict, keep
   safe + note
2 * 8ab4aeb merge feature-b: resolve conflict, keep safe + note
3 |\
4 | * aa6594e (feature-b) feature-b: set mode=fast
5 * | 5a08307 (feature-a) feature-a: set mode=safe
6 |/\
7 * d3a0388 init: mode=normal
```

Listing 12: 提交历史图 (merge-log.txt)

冲突产生的原因是 feature-a 与 feature-b 都修改了 config.txt 的同一行(mode=), Git 无法自动判断保留哪一个, 于是产生了冲突标记; 手工编辑时保留了 mode=safe 并增加 note=reviewed, 完成合并后工作区干净。

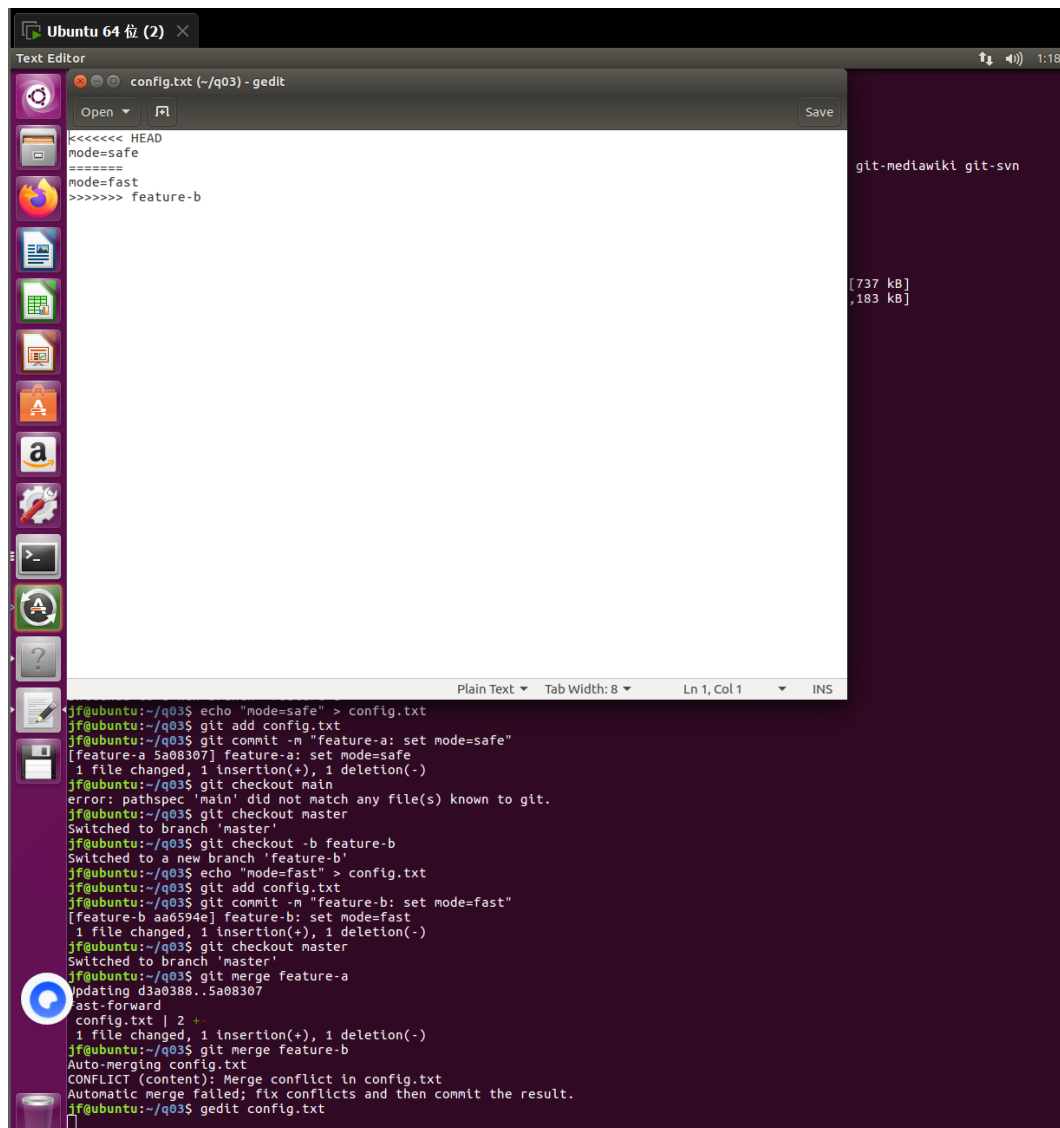


图 6: 第 3 题 gedit 中看到的冲突标记及产生冲突的过程

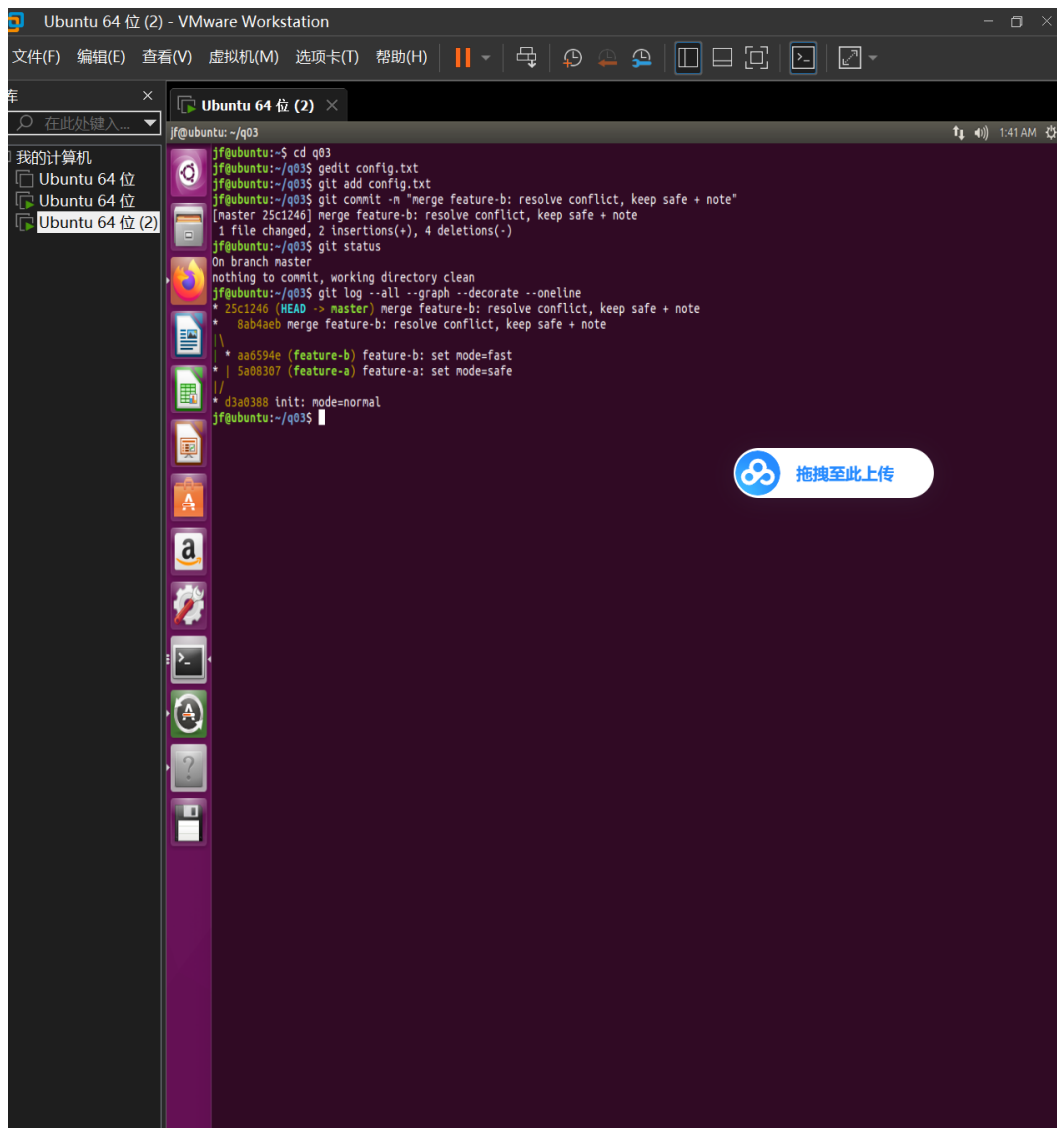


图 7: 第 3 题解决冲突并提交后的提交历史图

1.4 第 4 题: 修复并构建一页技术说明 (LaTeX 文档编辑)

题目描述

将给定模板保存为 `q04/report.tex`, 补全内容并从命令行构建 PDF:

1. 加入一个带编号和 label 的公式, 并在正文中使用 `ref` 或 `eqref` 引用;
2. 加入一个 2 行 2 列表格, 设置 `caption` 和 `label`, 并在正文中引用该表;
3. 使用 `latexmk -pdf -halt-on-error` 或 `tectonic` 生成 `report.pdf`;
4. 确认 PDF 能打开, 且交叉引用不显示 “??”。

作答过程与关键代码

```
1 \documentclass{article}
2 \usepackage{amsmath}
3 \begin{document}
4 \title{Tool Report}
5 \author{jf-25020007030}
6 \maketitle
7
8 \section{Result}
9 The calculation formula is shown in Equation~\eqref{eq:test_formula}
10
11 \begin{equation}
12 E=mc^2
13 \label{eq:test_formula}
14 \end{equation}
15
16 Table~\ref{tab:data_table}
17 \begin{table}[h]
18 \centering
19 \caption{Experimental comparison data}
20 \label{tab:data_table}
21 \begin{tabular}{|c|c|}
22 \hline
23 A & B \\
24 \hline
25 10 & 20 \\
26 \hline
27 30 & 40 \\
28 \hline
29 \end{tabular}
30 \end{table}
31
32 Equation~\eqref{eq:test_formula} is mass-energy relation;
33 Table~\ref{tab:data_table} records sample measurement
34 \end{document}
```

Listing 13: report.tex 补全后的关键内容

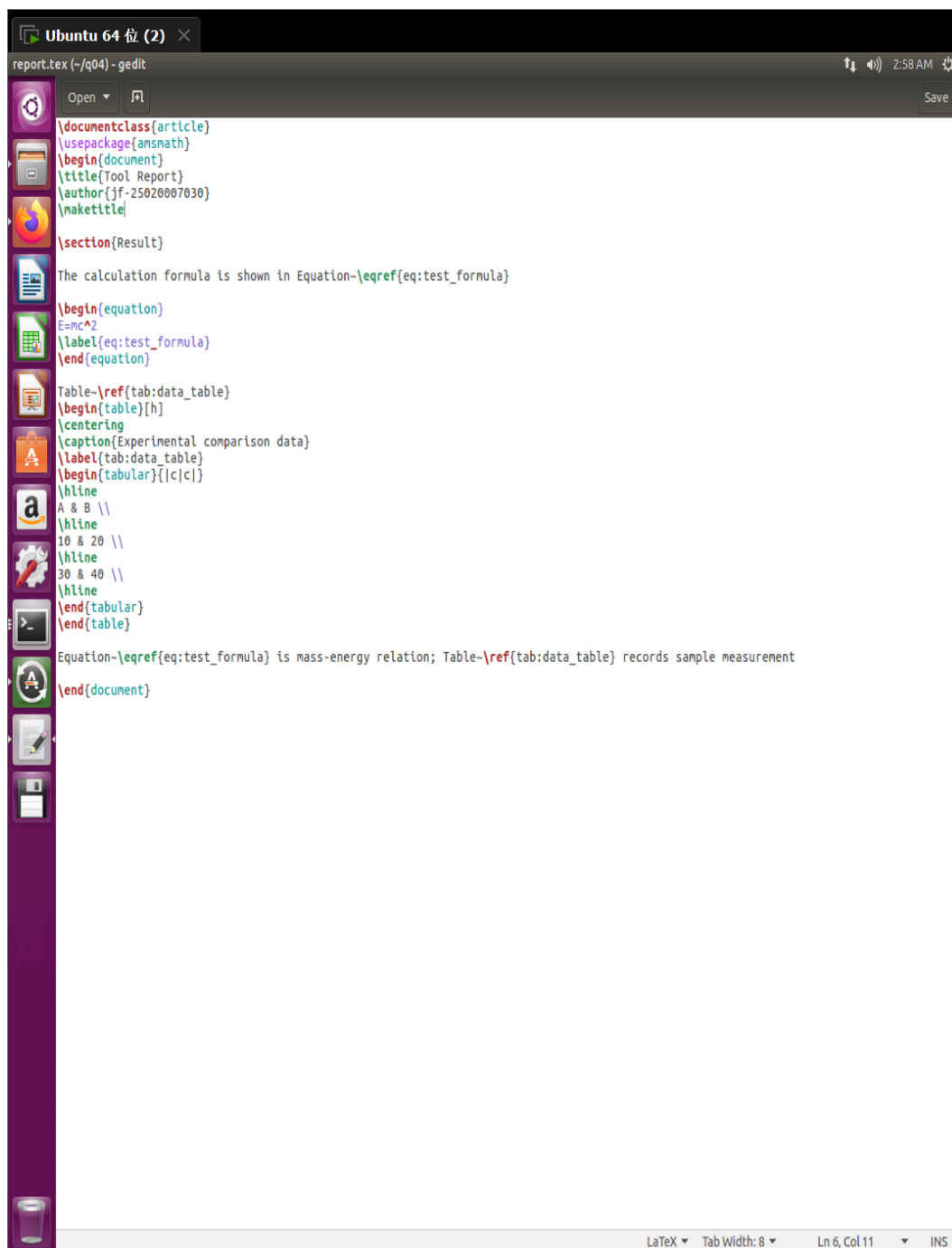
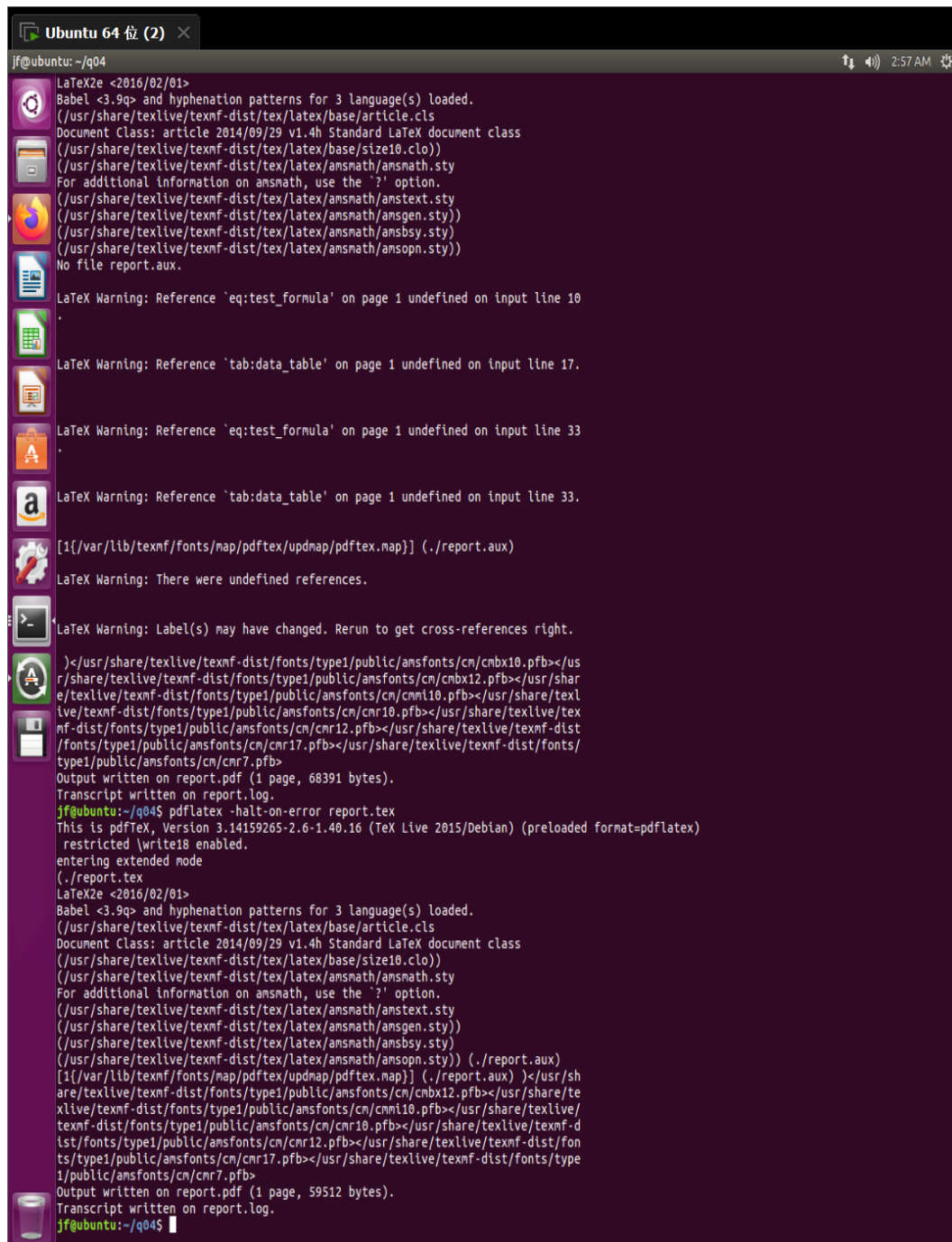


图 8: 第 4 题用 gedit 编辑 report.tex 的作答过程

运行结果

使用 `latexmk -pdf -halt-on-error report.tex` 编译成功, 生成了 `report.pdf`。PDF 可正常打开, 正文中公式引用显示为“(1)”、表格引用显示为“1”, 交叉引用不再出现“??”, 说明
label 与
eqref/
texttt

ref 对应关系正确。



```
Ubuntu 64 位 (2) x
jf@ubuntu: ~/q04
LaTeX2e <2016/02/01>
Babel <3.9q> and hyphenation patterns for 3 language(s) loaded.
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2014/09/29 v1.4h Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansdtext.sty
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansgen.sty))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansbsy.sty)
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansopn.sty))
No file report.aux.

LaTeX Warning: Reference 'eq:test_formula' on page 1 undefined on input line 10
.

LaTeX Warning: Reference 'tab:data_table' on page 1 undefined on input line 17.

LaTeX Warning: Reference 'eq:test_formula' on page 1 undefined on input line 33
.

LaTeX Warning: Reference 'tab:data_table' on page 1 undefined on input line 33.

[1{/var/lib/texmf/fonts/nap/psftex/updmap/psftex.map}] (./report.aux)
LaTeX Warning: There were undefined references.

LaTeX Warning: Label(s) may have changed. Rerun to get cross-references right.

) </usr/share/texlive/texmf-dist/fonts/type1/public/ansfonts/cn/cmbx10.pfb></usr/
r/share/texlive/texmf-dist/fonts/type1/public/ansfonts/cn/cmbx12.pfb></usr/shar
e/texlive/texmf-dist/fonts/type1/public/ansfonts/cn/cmm10.pfb></usr/share/texl
ive/texmf-dist/fonts/type1/public/ansfonts/cn/cmr10.pfb></usr/share/texlive/tex
mf-dist/fonts/type1/public/ansfonts/cn/cmr12.pfb></usr/share/texlive/texmf-dist
/fonts/type1/public/ansfonts/cn/cmr17.pfb></usr/share/texlive/texmf-dist/fonts/
type1/public/ansfonts/cn/cmr7.pfb>
Output written on report.pdf (1 page, 68391 bytes).
Transcript written on report.log.
jf@ubuntu:~/q04$ pdflatex -halt-on-error report.tex
This is pdfTeX, Version 3.14159265-2.6-1.40.16 (TeX Live 2015/Debian) (preloaded format=pdflatex)
restricted \write18 enabled.
entering extended mode
(./report.tex
LaTeX2e <2016/02/01>
Babel <3.9q> and hyphenation patterns for 3 language(s) loaded.
(/usr/share/texlive/texmf-dist/tex/latex/base/article.cls
Document Class: article 2014/09/29 v1.4h Standard LaTeX document class
(/usr/share/texlive/texmf-dist/tex/latex/base/size10.clo))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansdtext.sty
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansgen.sty))
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansbsy.sty)
(/usr/share/texlive/texmf-dist/tex/latex/amsmath/ansopn.sty)) (./report.aux)
[1{/var/lib/texmf/fonts/nap/psftex/updmap/psftex.map}] (./report.aux) )</usr/sh
are/texlive/texmf-dist/fonts/type1/public/ansfonts/cn/cmbx12.pfb></usr/share/te
xlive/texmf-dist/fonts/type1/public/ansfonts/cn/cmm10.pfb></usr/share/texlive/
texmf-dist/fonts/type1/public/ansfonts/cn/cmr10.pfb></usr/share/texlive/texmf-d
ist/fonts/type1/public/ansfonts/cn/cmr12.pfb></usr/share/texlive/texmf-dist/fon
ts/type1/public/ansfonts/cn/cmr17.pfb></usr/share/texlive/texmf-dist/fon
ts/type1/public/ansfonts/cn/cmr7.pfb>
Output written on report.pdf (1 page, 59512 bytes).
Transcript written on report.log.
jf@ubuntu:~/q04$
```

图 9: 第 4 题 pdflatex 编译 report.tex 生成 report.pdf

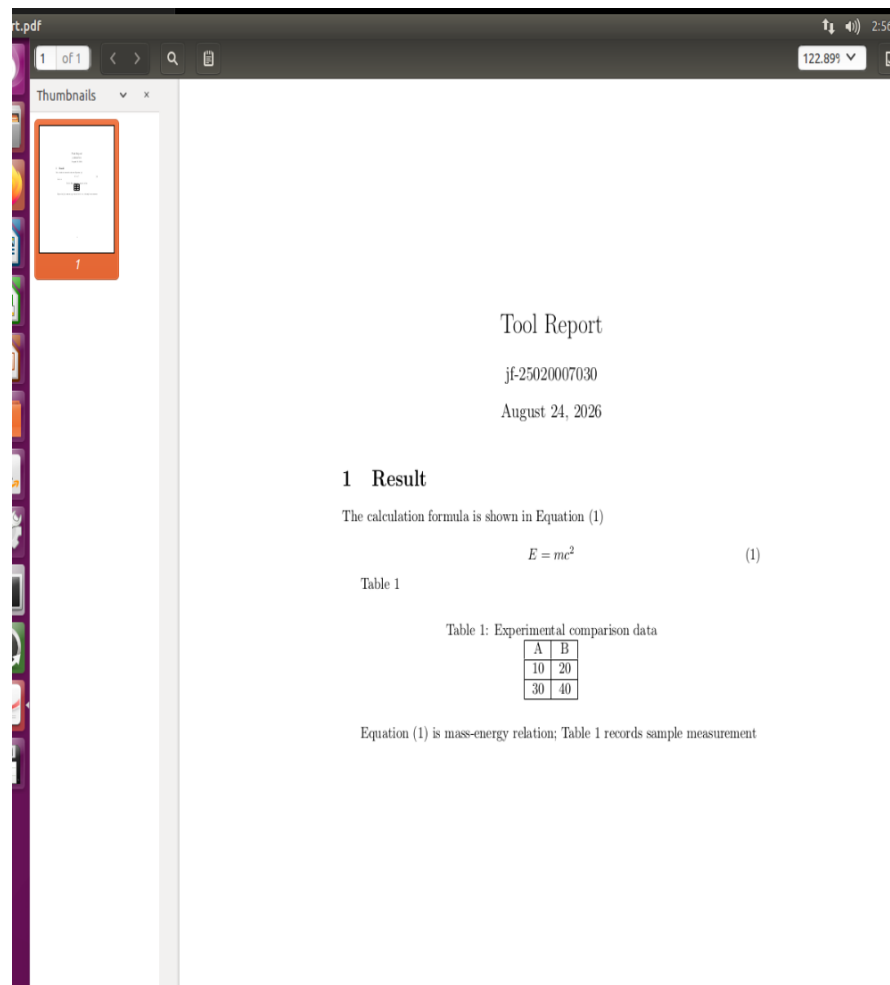


图 10: 第 4 题编译生成的 report.pdf 成品

2 课后练习内容和结果

课后共完成 11 个实例练习，按「题目—作答—结果」记录如下。按照课程要求，练习内容均已分步提交到个人 GitHub 仓库，并禁止单次全量提交。

2.1 Shell 基础与文件系统练习

2.1.1 第 2 题：ls 的 -l 选项 (flag) 作用是什么？

题目：运行 `ls -l /` 并观察输出，每一行最前面的 10 个字符分别代表什么？

2. ls 的 -l 选项 (flag) 作用是什么？运行 `ls -l /` 并观察输出。每一行最前面的 10 个字符分别代表什么？（提示：man ls）

图 11: 第 2 题题目截图

作答：-l 选项以长格式输出文件信息。每行最前面 10 个字符含义如下（以 -rw-r--r-- 为例）：

第 1 个字符 文件类型：- 普通文件、d 目录、l 符号链接、b/c 块/字符设备等

第 2-4 个字 属主（user）的读、写、执行权限（r/w/x，无权限为 -）
符

第 5-7 个字 属组（group）的读、写、执行权限
符

第 8-10 个 其他用户（other）的读、写、执行权限
字符

表 2: ls -l 前 10 个字符含义

结果：运行 `ls -l /` 后，每行形如 `drwxr-xr-x 1 root root 4096 ...`，其中 `drwxr-xr-x` 表明该目录属主可读可写可执行、属组和其他用户可读可执行。可通过 `man ls` 查看帮助确认。

```

jlf@ubuntu:~$ ls -l /
drwxr-xr-x 135 root root 12288 Aug 25 18:48 etc
drwxr-xr-x  3 root root  4096 Aug 24 00:03 home
lrwxrwxrwx  1 root root   34 Aug 24 06:57 initrd.img -> boot/initrd.img-4.15.0-142-generic
lrwxrwxrwx  1 root root   34 Aug 24 07:49 initrd.img.old -> boot/initrd.img-4.15.0-112-generic
drwxr-xr-x 22 root root  4096 Aug 24 00:04 lib
drwxr-xr-x  2 root root  4096 Aug 24 06:57 lib64
drwx----- 2 root root 16384 Aug 24 07:49 lost+found
drwxr-xr-x  3 root root  4096 Aug  6 2020 media
drwxr-xr-x  3 root root  4096 Aug 24 23:16 mnt
drwxr-xr-x  2 root root  4096 Aug 24 00:05 opt
dr-xr-xr-x 238 root root    0 Aug 26 02:11 proc
drwx----- 3 root root  4096 Aug  6 2020 root
drwxr-xr-x 25 root root   800 Aug 26 04:02 run
drwxr-xr-x  2 root root 12288 Aug 25 18:48/sbin
drwxr-xr-x  2 root root  4096 Aug 24 00:04 snap
drwxr-xr-x  2 root root  4096 Aug  6 2020 srv
dr-xr-xr-x 13 root root    0 Aug 26 02:11 sys
drwxrwxrwt 13 root root  4096 Aug 26 04:02 tmp
drwxr-xr-x 12 root root  4096 Aug  6 2020 usr
drwxr-xr-x 14 root root  4096 Aug  6 2020 var
lrwxrwxrwx  1 root root   31 Aug 24 06:57 vmlinuz -> boot/vmlinuz-4.15.0-142-generic
lrwxrwxrwx  1 root root   31 Aug 24 06:57 vmlinuz.old -> boot/vmlinuz-4.15.0-112-generic
jlf@ubuntu:~$ man ls
  
```

图 12: 运行 `ls -l /` 的结果

2.1.2 第 5 题：Shell 标准流与重定向

题目：Shell 有三条标准流 `stdin(0)`、`stdout(1)`、`stderr(2)`。运行 `ls /nonexistent /tmp`，把 `stdout` 和 `stderr` 分别重定向到两个文件。你将如何把两者都重定向到同一个文件？

5. Shell 有三条标准流：`stdin (0)`、`stdout (1)`、`stderr (2)`。运行 `ls /nonexistent /tmp`，把 `stdout` 和 `stderr` 分别重定向到两个文件。你将如何把两者都重定向到同一个文件？参见 [Redirections](#)。

图 13: 第 5 题题目截图

作答：

```
1 ls /nonexistent /tmp > out.txt 2> err.txt
2 # 将 stdout 与 stderr 都重定向到同一个文件
3 ls /nonexistent /tmp > all.txt 2>&1
```

Listing 14: 分别重定向 `stdout` 与 `stderr`

结果：

- `out.txt`：只包含 `/tmp` 的正常列表输出 (`stdout`)；
- `err.txt`：只包含 `ls: cannot access '/nonexistent': No such file or directory` (`stderr`)；
- `all.txt`：使用 `2>&1` 将 `stderr` 合并到 `stdout` 后，两类内容写入同一文件。

```
jf@ubuntu:~$ ls /nonexistent /tmp > out.txt 2> err.txt
jf@ubuntu:~$ ls /nonexistent /tmp > all.txt 2>&1
bash: syntax error near unexpected token `2'
jf@ubuntu:~$ ls /nonexistent /tmp > all.txt 2>&1
jf@ubuntu:~$ cat all.txt
ls: cannot access '/nonexistent': No such file or directory
/tmp:
config-err-UYyf5A
systemd-private-fb7cee477449475c954db269c989c2cb-color.service-vZAHya
systemd-private-fb7cee477449475c954db269c989c2cb-fwupd.service-PmDznX
systemd-private-fb7cee477449475c954db269c989c2cb-rtkit-daemon.service-V9rbQ4
systemd-private-fb7cee477449475c954db269c989c2cb-systemd-timesyncd.service-jjM8g
8
unity_support_test.0
VMwareDnD
vmware-root
jf@ubuntu:~$
```

图 14: 标准流重定向的运行结果

2.1.3 第 8 题：检查文件是否存在的脚本

题目：写一个脚本，接收文件名参数 (\$1)，用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在，并根据结果输出不同提示。

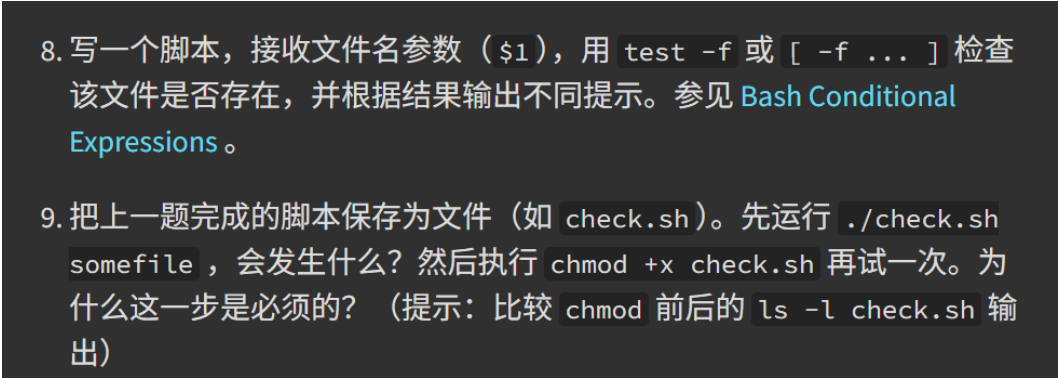
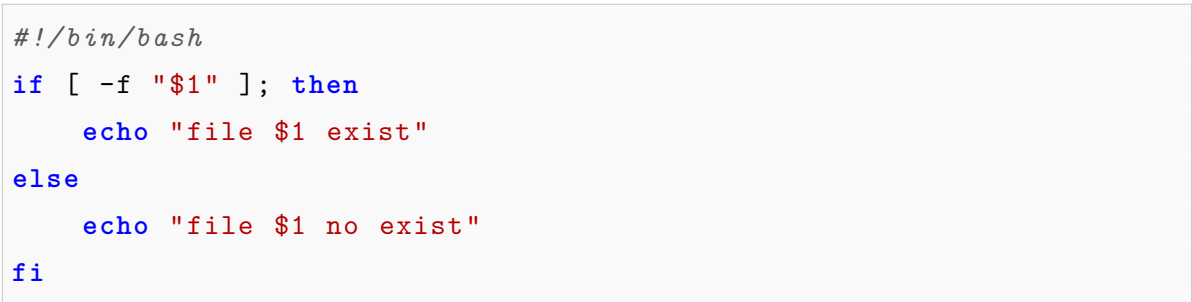
- 
8. 写一个脚本，接收文件名参数 (\$1)，用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在，并根据结果输出不同提示。参见 [Bash Conditional Expressions](#)。
9. 把上一题完成的脚本保存为文件 (如 `check.sh`)。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？(提示：比较 `chmod` 前后的 `ls -l check.sh` 输出)

图 15: 第 8、9 题题目截图

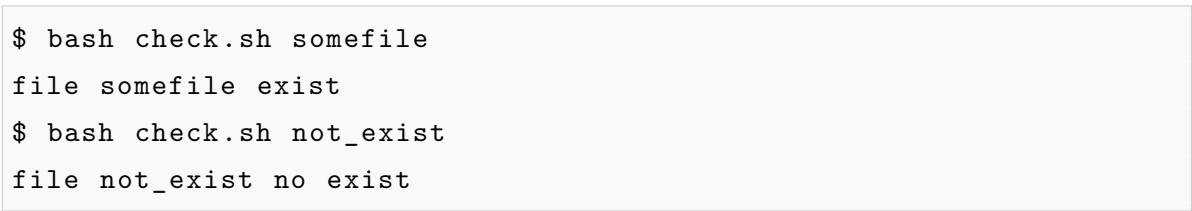
作答：



```
1 #!/bin/bash
2 if [ -f "$1" ]; then
3     echo "file $1 exist"
4 else
5     echo "file $1 no exist"
6 fi
```

Listing 15: check.sh

结果：



```
1 $ bash check.sh somefile
2 file somefile exist
3 $ bash check.sh not_exist
4 file not_exist no exist
```

Listing 16: 运行结果

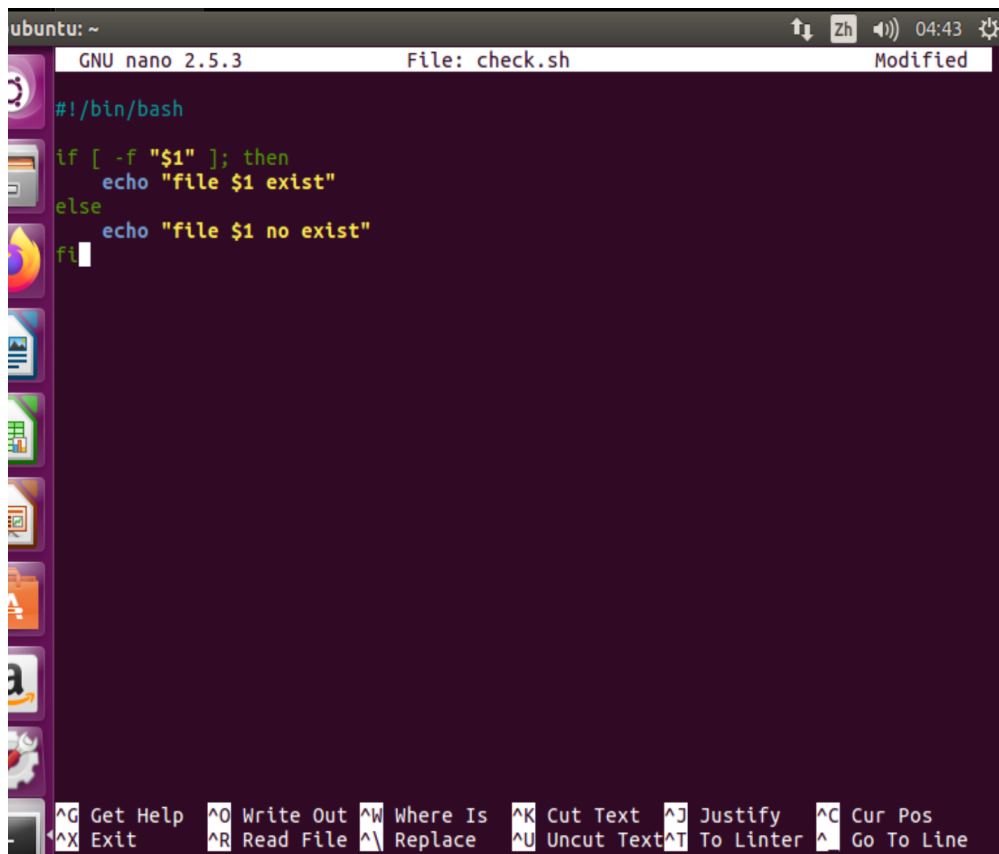


图 16: 用 nano 编写 check.sh 脚本

2.1.4 第 9 题: 为脚本添加可执行权限 (chmod)

题目: 把上一题脚本保存为 check.sh。先运行 ./check.sh somefile, 会发生什么? 然后执行 chmod +x check.sh 再试一次。为什么这一步是必须的?

作答: 直接运行 ./check.sh 时, 由于文件没有可执行权限, Shell 会报 Permission denied; 执行 chmod +x check.sh 后再运行则正常。权限是操作系统对文件执行的访问控制, 只有具备 x (execute) 权限的文件才能被直接执行; chmod +x 即为属主、属组和其他用户添加执行权限。

结果:

```
1 $ ls -l check.sh          # 修改前
2 -rw-r--r-- 1 jf jf ... check.sh
3 $ ./check.sh somefile
4 bash: ./check.sh: Permission denied
5 $ chmod +x check.sh
6 $ ls -l check.sh          # 修改后, 属主出现 x 权限
7 -rwxr-xr-x 1 jf jf ... check.sh
8 $ ./check.sh somefile
9 file somefile exist
```

Listing 17: chmod 前后对比

```
jf@ubuntu:~$ nano check.sh
jf@ubuntu:~$ ls -l check.sh
-rw-rw-r-- 1 jf jf 97 Aug 26 04:45 check.sh
jf@ubuntu:~$ ./check.sh somefile
bash: ./check.sh: Permission denied
jf@ubuntu:~$ chmod +x check.sh
jf@ubuntu:~$ ls -l check.sh
-rwxrwxr-x 1 jf jf 97 Aug 26 04:45 check.sh
jf@ubuntu:~$ ./check.sh somefile
file somefile no exist
jf@ubuntu:~$ touch somefile
jf@ubuntu:~$ ./check.sh somefile
file somefile exist
jf@ubuntu:~$
```

图 17: chmod +x 前后对比及运行结果

2.1.5 第 10 题: set -x 选项的作用

题目: 在脚本的 set 选项里加入 -x 会发生什么? 写个简单脚本试试并观察输出。

10. 在脚本的 set 选项 (flag) 里加入 -x 会发生什么? 写个简单脚本试试并观察输出。参见 [The Set Builtin](#)。

图 18: 第 10 题题目截图

作答: set -x 开启命令跟踪 (xtrace), 脚本执行时会把每条将要执行的命令 (展开后的形式, 前缀 +) 打印到 stderr, 便于调试。

```
1 #!/bin/bash
2 set -x
3 echo "hello"
4 ls /nonexistent
```

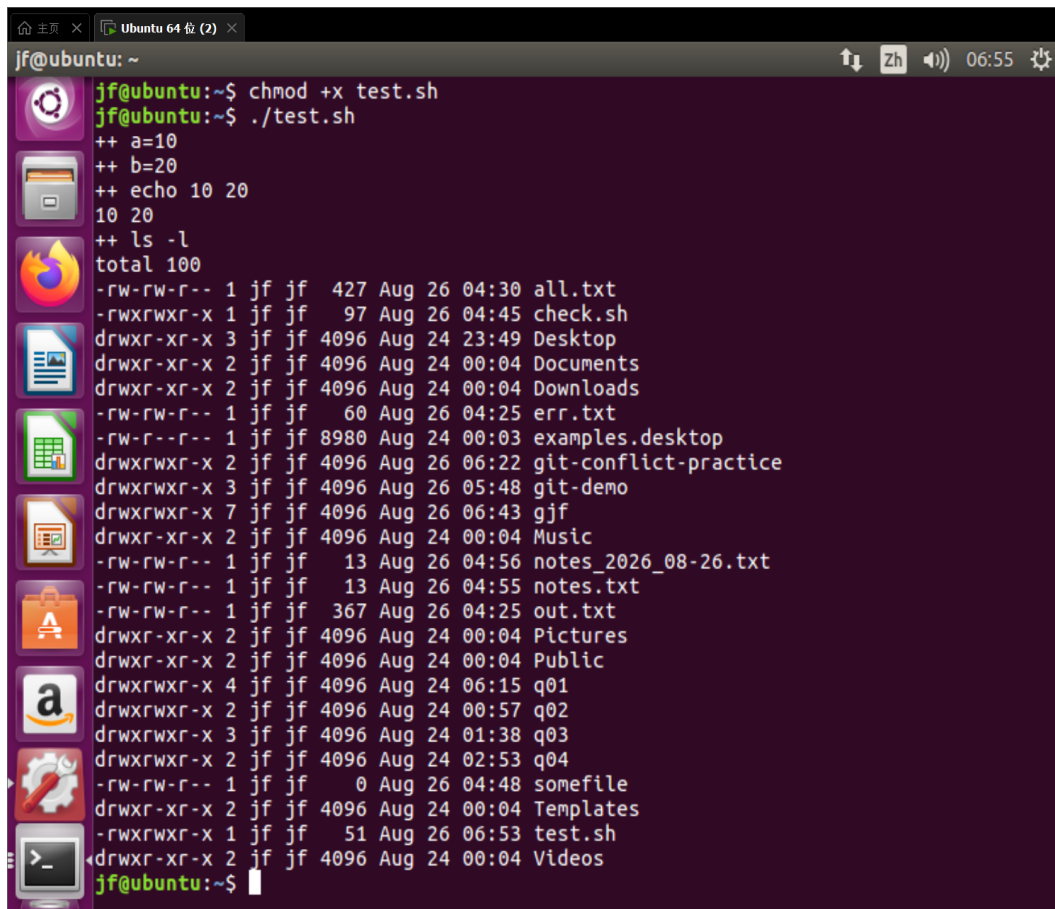
Listing 18: 测试 set -x 的脚本

结果: 运行后输出类似:

```
1 + echo hello
2 hello
3 + ls /nonexistent
4 ls: cannot access '/nonexistent': No such file or directory
```

Listing 19: set -x 输出

可以看到每条命令都被打印出来（前面带 +），方便定位脚本执行到哪一步出错。参见 Bash 的 The Set Builtin。



```
jf@ubuntu: ~  
jf@ubuntu:~$ chmod +x test.sh  
jf@ubuntu:~$ ./test.sh  
++ a=10  
++ b=20  
++ echo 10 20  
10 20  
++ ls -l  
total 100  
-rw-rw-r-- 1 jf jf 427 Aug 26 04:30 all.txt  
-rwxrwxr-x 1 jf jf 97 Aug 26 04:45 check.sh  
drwxr-xr-x 3 jf jf 4096 Aug 24 23:49 Desktop  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Documents  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Downloads  
-rw-rw-r-- 1 jf jf 60 Aug 26 04:25 err.txt  
-rw-r--r-- 1 jf jf 8980 Aug 24 00:03 examples.desktop  
drwxrwxr-x 2 jf jf 4096 Aug 26 06:22 git-conflict-practice  
drwxrwxr-x 3 jf jf 4096 Aug 26 05:48 git-demo  
drwxrwxr-x 7 jf jf 4096 Aug 26 06:43 gjf  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Music  
-rw-rw-r-- 1 jf jf 13 Aug 26 04:56 notes_2026_08-26.txt  
-rw-rw-r-- 1 jf jf 13 Aug 26 04:55 notes.txt  
-rw-rw-r-- 1 jf jf 367 Aug 26 04:25 out.txt  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Pictures  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Public  
drwxrwxr-x 4 jf jf 4096 Aug 24 06:15 q01  
drwxrwxr-x 2 jf jf 4096 Aug 24 00:57 q02  
drwxrwxr-x 3 jf jf 4096 Aug 24 01:38 q03  
drwxrwxr-x 2 jf jf 4096 Aug 24 02:53 q04  
-rw-rw-r-- 1 jf jf 0 Aug 26 04:48 somefile  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Templates  
-rwxrwxr-x 1 jf jf 51 Aug 26 06:53 test.sh  
drwxr-xr-x 2 jf jf 4096 Aug 24 00:04 Videos  
jf@ubuntu:~$
```

图 19: 运行带 set -x 的脚本，逐条追踪命令

2.1.6 第 11 题：带当天日期的备份文件名

题目：写一条命令，把文件复制为带当天日期的备份文件名（例如 notes.txt → notes_2026-01-12.txt）。

11. 写一条命令，把文件复制为带当天日期的备份文件名（例如 notes.txt → notes_2026-01-12.txt）。（提示：\$(date +%Y-%m-%d)）参见 [Command Substitution](#)。

图 20: 第 11 题题目截图

作答：使用命令替换 \$(date +%Y-%m-%d) 生成当天日期：

```
1 cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

Listing 20: 带日期备份命令

结果: 2026 年 8 月 26 日执行后生成 notes_2026_08-26.txt, 内容与 notes.txt 一致 (test content)。

```
jff@ubuntu:~$ echo "test content" > notes.txt
jff@ubuntu:~$ cp notes.txt note_$(date +%Y-%m-%d).txt
cp: cannot stat 'notes.txt': No such file or directory
jff@ubuntu:~$ cp notes.txt notes_$(date +%Y-%m-%d).txt
jff@ubuntu:~$ ls
all.txt      Downloads      notes_2026_08-26.txt  Public  q04
check.sh     err.txt        notes.txt             q01     somefile
Desktop      examples.desktop out.txt               q02     Templates
Documents    Music          Pictures              q03     Videos
jff@ubuntu:~$
```

图 21: 带当天日期的备份文件名运行结果

2.2 Git 版本控制练习

2.2.1 第 3 题: 从 Git 历史中删除误提交的文件

题目: 学习 Git 时常见的错误是把不应由 Git 管理的大文件或敏感信息提交进去。尝试添加一个文件、做几次提交, 然后从历史记录中删除该文件 (而不仅是删掉最新提交)。

3. One common mistake when learning Git is to commit large files that should not be managed by Git or adding sensitive information. Try adding a file to a repository, making some commits and then deleting that file from *history* (not just the latest commit). You may want to look at [this](#).

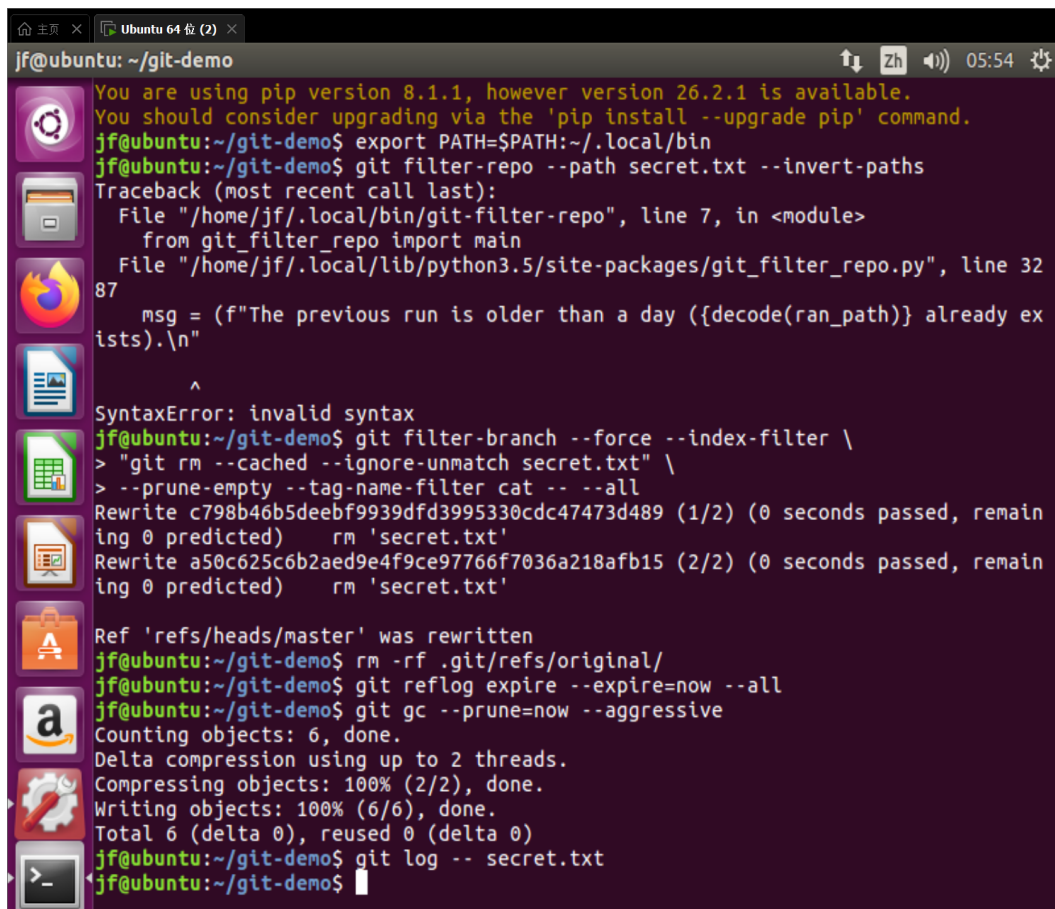
图 22: Git 第 3 题题目截图

作答: 将文件加入仓库并提交后, 若需从历史中彻底删除, 可使用 `git filter-branch` 或 `git rebase -i` 改写历史:

```
1 git filter-branch --force --index-filter \
2   'git rm --cached --ignore-unmatch secret.txt' --prune-empty -- --
   all
3 # 清理残留对象
4 git for-each-ref --format='%(refname)' refs/original/ | xargs -n1
   git update-ref -d
5 git reflog expire --expire=now --all && git gc --prune=now --
   aggressive
```

Listing 21: 从历史删除文件

结果：改写后的提交历史中不再包含该文件；此操作通常用于删除敏感信息或超大文件，但会改写历史哈希，仅适用于尚未推送或可接受强制推送的场景。



```
jf@ubuntu: ~/git-demo
You are using pip version 8.1.1, however version 26.2.1 is available.
You should consider upgrading via the 'pip install --upgrade pip' command.
jf@ubuntu:~/git-demo$ export PATH=$PATH:~/local/bin
jf@ubuntu:~/git-demo$ git filter-repo --path secret.txt --invert-paths
Traceback (most recent call last):
  File "/home/jf/.local/bin/git-filter-repo", line 7, in <module>
    from git_filter_repo import main
  File "/home/jf/.local/lib/python3.5/site-packages/git_filter_repo.py", line 32
    87
    msg = (f"The previous run is older than a day ({decode(ran_path)} already exists).\n"
    ^
SyntaxError: invalid syntax
jf@ubuntu:~/git-demo$ git filter-branch --force --index-filter \
> "git rm --cached --ignore-unmatch secret.txt" \
> --prune-empty --tag-name-filter cat -- --all
Rewrite c798b46b5deebf9939dfd3995330cdc47473d489 (1/2) (0 seconds passed, remain
ing 0 predicted)    rm 'secret.txt'
Rewrite a50c625c6b2aed9e4f9ce97766f7036a218afb15 (2/2) (0 seconds passed, remain
ing 0 predicted)    rm 'secret.txt'

Ref 'refs/heads/master' was rewritten
jf@ubuntu:~/git-demo$ rm -rf .git/refs/original/
jf@ubuntu:~/git-demo$ git reflog expire --expire=now --all
jf@ubuntu:~/git-demo$ git gc --prune=now --aggressive
Counting objects: 6, done.
Delta compression using up to 2 threads.
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), done.
Total 6 (delta 0), reused 0 (delta 0)
jf@ubuntu:~/git-demo$ git log -- secret.txt
jf@ubuntu:~/git-demo$
```

图 23: 用 git filter-branch 从历史中删除 secret.txt

2.2.2 第 4 题: git stash 暂存未完成的工作

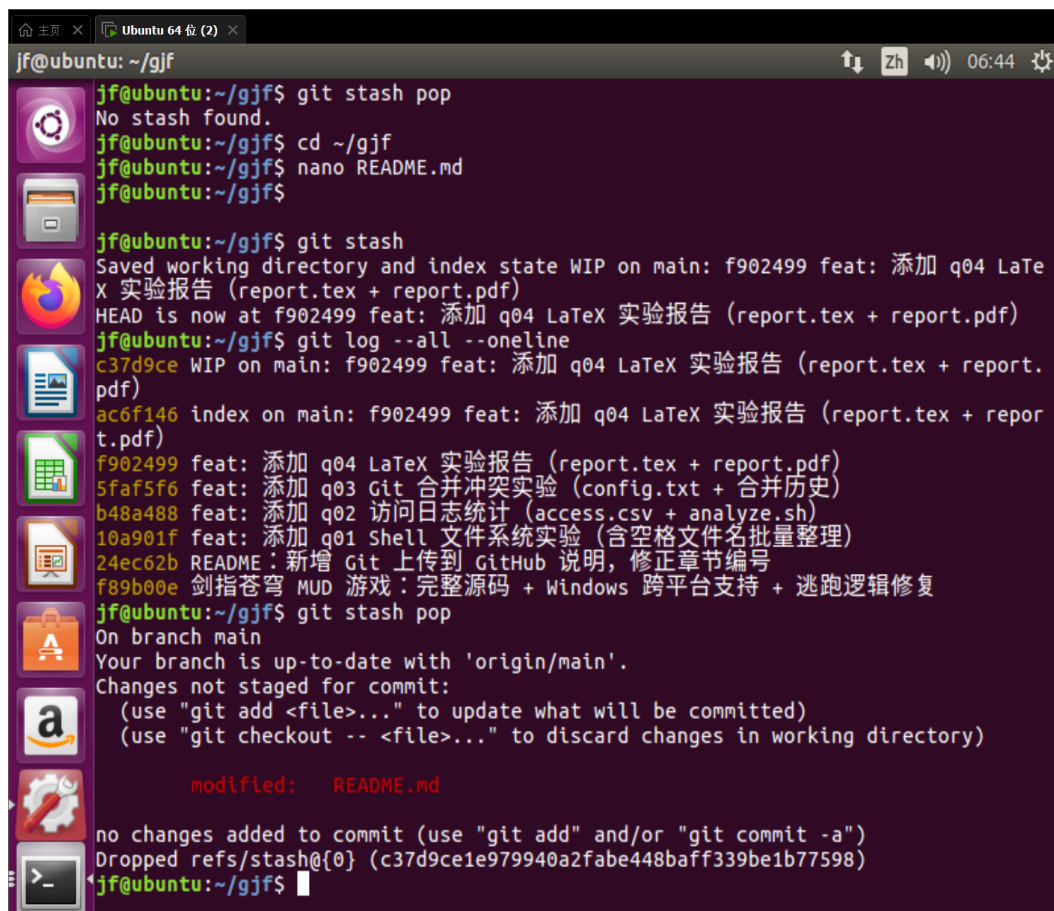
题目：从 GitHub 克隆某个仓库，修改其中一个已有文件。做 git stash 会发生什么？运行 git log --all --oneline 会看到什么？运行 git stash pop 来撤销刚才的操作。这个操作在什么场景下有用？

```
4. Clone some repository from GitHub, and modify one of its existing files. What happens when you do ? What do you see when running ? Run to undo what you did with . In what scenario might this be useful? git stash git log --all --oneline git stash pop git stash
```

图 24: Git 第 4 题题目截图

作答：git stash 会把工作区的未提交修改临时保存起来，使工作区恢复干净；git log --all --oneline 会看到当前分支及 stash 创建的相关提交（stash 也会记录一条 WIP 提交）；git stash pop 把暂存的修改恢复到工作区。

结果：该操作在需要临时切换分支、却不想丢弃或提交当前半成品修改时非常有用，例如接到紧急任务需切走时，可先 `git stash`，处理完再 `git stash pop` 恢复。



```
jf@ubuntu: ~/gjf
jf@ubuntu:~/gjf$ git stash pop
No stash found.
jf@ubuntu:~/gjf$ cd ~/gjf
jf@ubuntu:~/gjf$ nano README.md
jf@ubuntu:~/gjf$

jf@ubuntu:~/gjf$ git stash
Saved working directory and index state WIP on main: f902499 feat: 添加 q04 LaTeX 实验报告 (report.tex + report.pdf)
HEAD is now at f902499 feat: 添加 q04 LaTeX 实验报告 (report.tex + report.pdf)
jf@ubuntu:~/gjf$ git log --all --oneline
c37d9ce WIP on main: f902499 feat: 添加 q04 LaTeX 实验报告 (report.tex + report.pdf)
ac6f146 index on main: f902499 feat: 添加 q04 LaTeX 实验报告 (report.tex + report.pdf)
f902499 feat: 添加 q04 LaTeX 实验报告 (report.tex + report.pdf)
5faf5f6 feat: 添加 q03 Git 合并冲突实验 (config.txt + 合并历史)
b48a488 feat: 添加 q02 访问日志统计 (access.csv + analyze.sh)
10a901f feat: 添加 q01 Shell 文件系统实验 (含空格文件名批量整理)
24ec62b README: 新增 Git 上传到 GitHub 说明, 修正章节编号
f89b00e 剑指苍穹 MUD 游戏: 完整源码 + Windows 跨平台支持 + 逃跑逻辑修复
jf@ubuntu:~/gjf$ git stash pop
On branch main
Your branch is up-to-date with 'origin/main'.
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (c37d9ce1e979940a2fabe448baff339be1b77598)
jf@ubuntu:~/gjf$
```

图 25: git stash 与 git stash pop 的完整过程

2.2.3 第 5 题：为 git log 配置别名

题目：Git 提供配置文件 `.gitconfig`。创建一个别名，使运行 `git graph` 时得到 `git log --all --graph --decorate --oneline` 的输出。

```
5. Like many command line tools, Git provides a configuration file (or dotfile) called . Create an alias in so that when you run , you get the output of . You can do this by directly editing the file, or you can use the command to add the alias. Information about git aliases can be found here. ~/.gitconfig ~/.gitconfig git graph git log --all --graph --decorate --oneline ~/.gitconfig git config
```

图 26: Git 第 5 题题目截图

作答：

```
1 git config --global alias.graph "log --all --graph --decorate --  
  oneline"  
2 # 之后即可用  
3 git graph
```

Listing 22: 配置 git 别名

结果：配置写入 `.gitconfig` 的 `[alias]` 段。之后 `git graph` 即可输出图形化的完整提交历史。

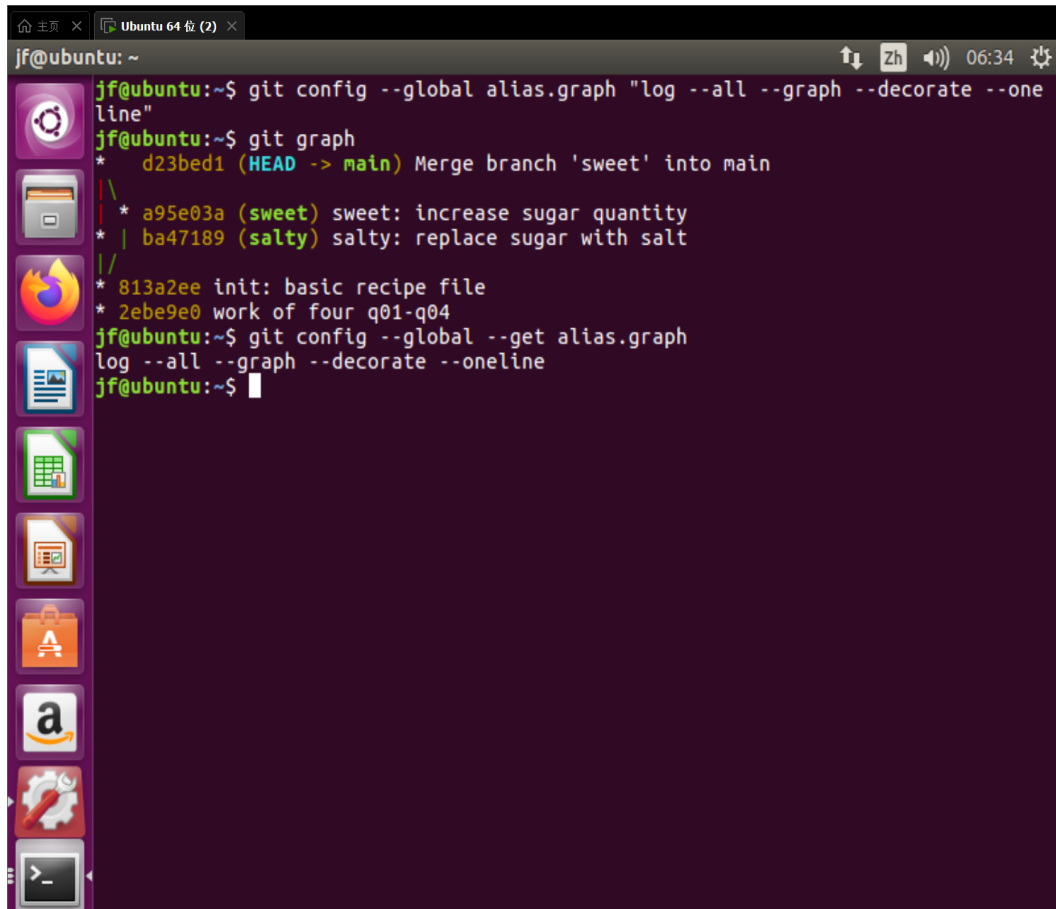
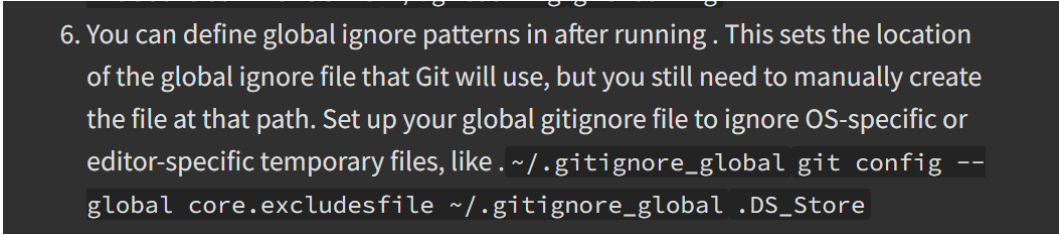


图 27: 配置 git graph 别名并运行

2.2.4 第 6 题：配置全局 gitignore

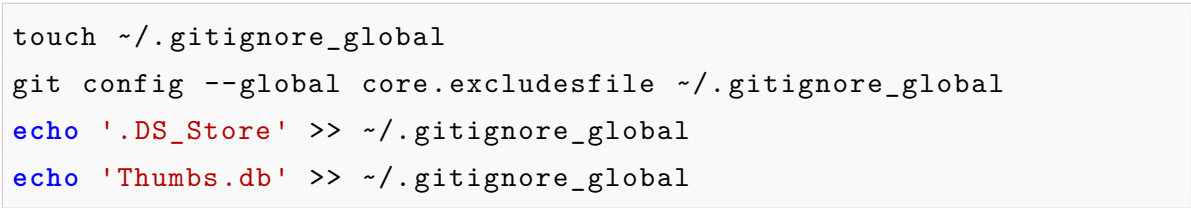
题目：通过 `git config --global core.excludesfile` 指定全局忽略文件路径，并手动创建该文件，忽略操作系统或编辑器产生的临时文件。

A screenshot of a text box containing a problem statement in English. The text describes how to define global ignore patterns in Git, mentioning the location of the global ignore file and the need to manually create it. It also provides a command to set up the global gitignore file to ignore OS-specific or editor-specific temporary files, with an example path for a .DS_Store file.

6. You can define global ignore patterns in after running . This sets the location of the global ignore file that Git will use, but you still need to manually create the file at that path. Set up your global gitignore file to ignore OS-specific or editor-specific temporary files, like `~/.gitignore_global` `git config --global core.excludesfile ~/.gitignore_global` `.DS_Store`

图 28: Git 第 6 题题目截图

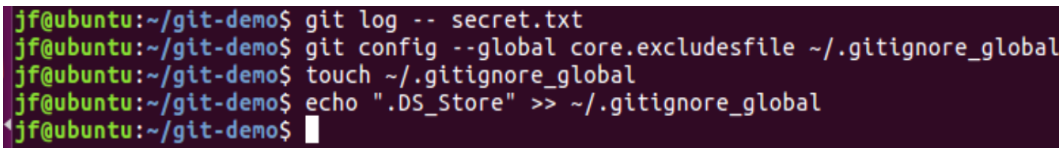
作答:

A screenshot of a terminal window showing four lines of shell commands. The first line creates a file named .gitignore_global in the home directory. The second line configures Git to use this file as the global excludesfile. The third and fourth lines use echo to append the strings '.DS_Store' and 'Thumbs.db' to the .gitignore_global file.

```
1 touch ~/.gitignore_global
2 git config --global core.excludesfile ~/.gitignore_global
3 echo '.DS_Store' >> ~/.gitignore_global
4 echo 'Thumbs.db' >> ~/.gitignore_global
```

Listing 23: 配置全局忽略文件

结果: 全局忽略文件生效, 所有仓库都会忽略 macOS/Windows 系统临时文件 (如.DS_Store、Thumbs.db), 避免误提交。

A screenshot of a terminal window showing a sequence of five commands. The first command runs 'git log -- secret.txt'. The second command configures Git to use a global excludesfile. The third command creates the .gitignore_global file. The fourth and fifth commands use echo to append '.DS_Store' to the .gitignore_global file.

```
jf@ubuntu:~/git-demo$ git log -- secret.txt
jf@ubuntu:~/git-demo$ git config --global core.excludesfile ~/.gitignore_global
jf@ubuntu:~/git-demo$ touch ~/.gitignore_global
jf@ubuntu:~/git-demo$ echo ".DS_Store" >> ~/.gitignore_global
jf@ubuntu:~/git-demo$
```

图 29: 配置全局 gitignore 文件

2.2.5 第 8 题: 模拟协作场景解决合并冲突

题目: 创建一个仓库和 `recipe.txt` (如一个简单的菜谱), 提交后创建 `salty` 和 `sweet` 两个分支; 分别在两个分支修改同一行并提交; 切回 `master` 依次合并两个分支触发冲突, 查看冲突标记含义, 编辑文件解决冲突并完成合并, 最后用 `git log --graph --oneline` 查看合并历史。

8. Practice resolving merge conflicts by simulating a collaborative scenario:
 1. Create a new repository with and create a file called with a few lines (e.g., a simple recipe). `git init recipe.txt`
 2. Commit it, then create two branches: and `git branch salty git branch sweet`
 3. In the branch, modify a line (e.g., change "1 cup sugar" to "1 cup salt") and commit. `salty`
 4. In the branch, modify the same line differently (e.g., change "1 cup sugar" to "2 cups sugar") and commit. `sweet`
 5. Now switch to and try , then . What happens? Look at the contents of - what do the , , and markers mean? `master git merge salty git merge sweet recipe.txt <<<<<< ===== >>>>>>`
 6. Resolve the conflict by editing the file to keep the content you want, removing the conflict markers, and completing the merge with and (or). Alternatively, try using to resolve the conflict with a graphical or terminal-based merge tool. `git add git commit git merge --continue git mergetool`
 7. Use to visualize the merge history you just created. `git log --graph --oneline`

图 30: Git 第 8 题题目截图

作答:

```

1 git init
2 # recipe.txt 中含 "1 cup sugar" 等行
3 git add recipe.txt && git commit -m "init recipe"
4 git branch salty && git branch sweet
5 git switch salty    # 将 "1 cup sugar" 改为 "1 cup salt" 并提交
6 git switch sweet    # 将 "1 cup sugar" 改为 "2 cups sugar" 并提交
7 git switch master
8 git merge salty
9 git merge sweet     # 冲突
10 # 编辑 recipe.txt 保留所需内容, 删除冲突标记
11 git add recipe.txt && git commit    # 或 git merge --continue
12 git log --graph --oneline

```

Listing 24: 制造并解决冲突

结果: 冲突标记 <<<<<< 到 ===== 之间是当前分支 (HEAD) 的内容, ===== 到 >>>>>> 之间是待合并分支的内容。解决冲突后的 `recipe.txt` 内容如下:

```

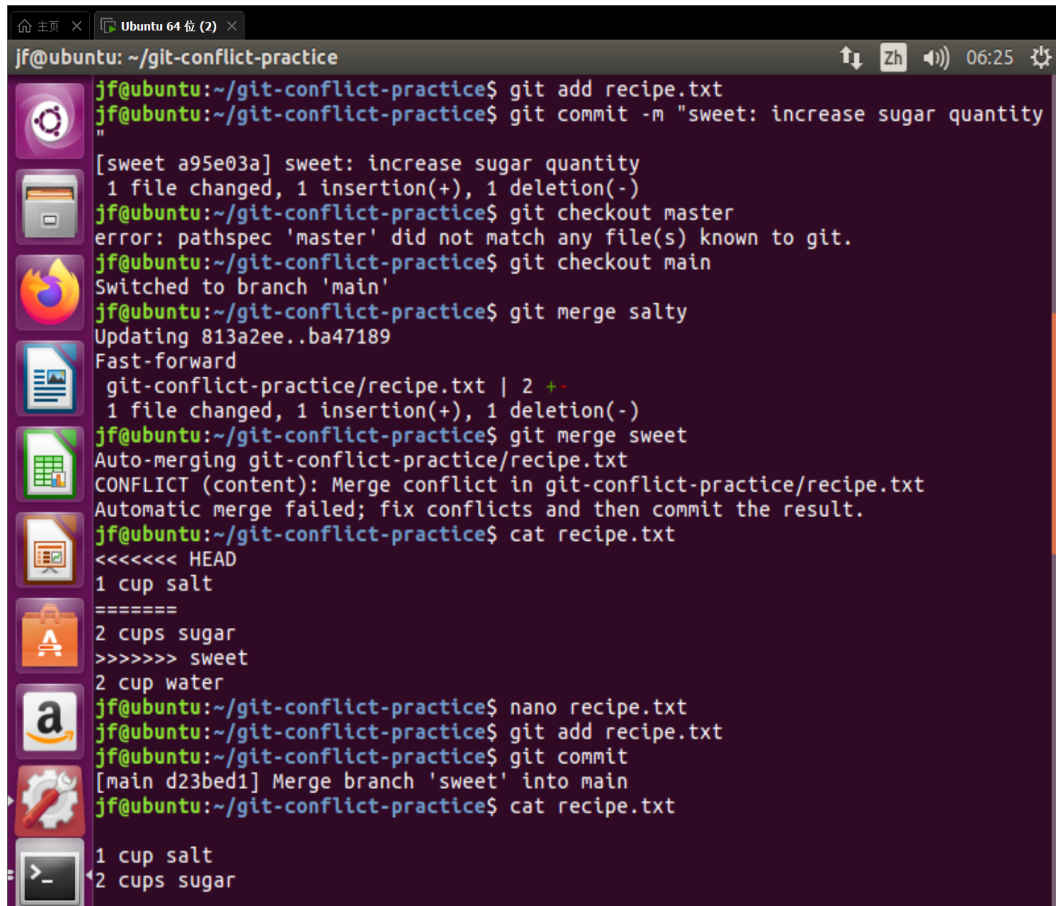
1 1 cup salt
2 2 cups sugar

```

3 2 cup water

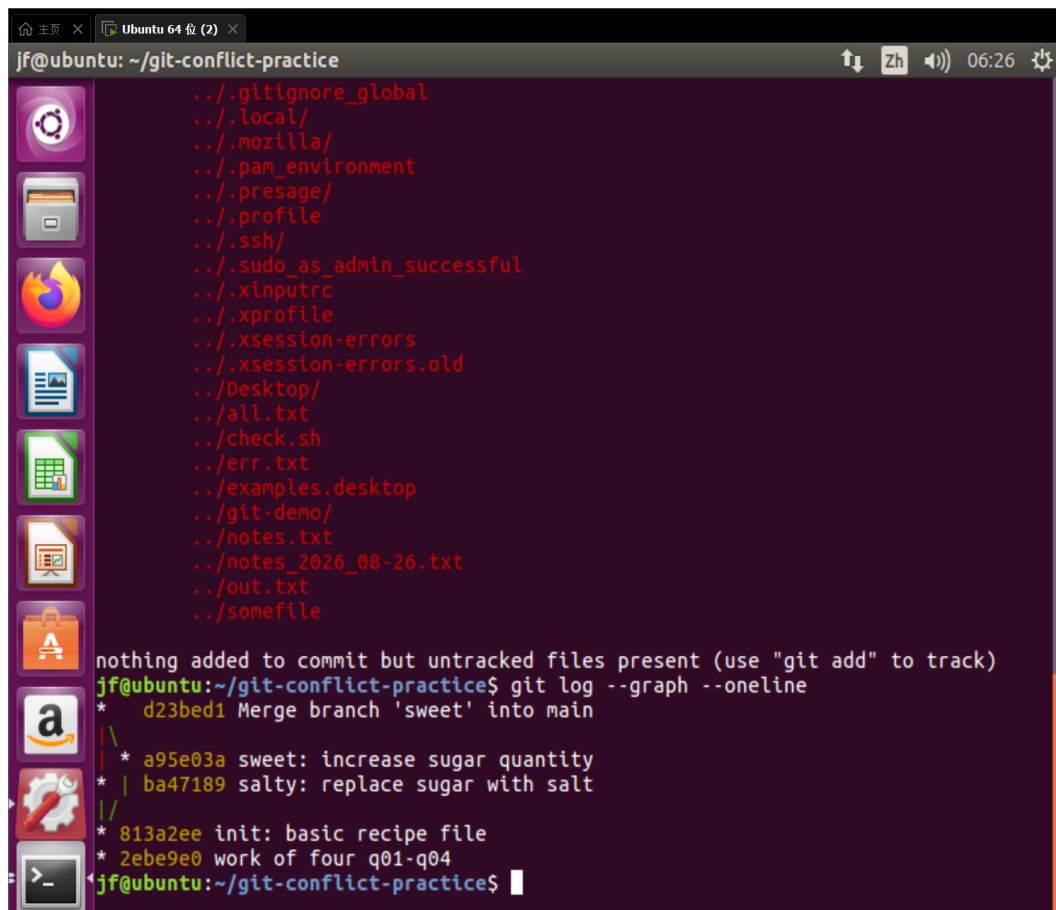
Listing 25: 解决冲突后的 recipe.txt

合并完成后用 `git log --graph --oneline` 可以看到分叉又合并的提交历史。



```
jf@ubuntu: ~/git-conflict-practice
jf@ubuntu:~/git-conflict-practice$ git add recipe.txt
jf@ubuntu:~/git-conflict-practice$ git commit -m "sweet: increase sugar quantity"
[sweet a95e03a] sweet: increase sugar quantity
1 file changed, 1 insertion(+), 1 deletion(-)
jf@ubuntu:~/git-conflict-practice$ git checkout master
error: pathspec 'master' did not match any file(s) known to git.
jf@ubuntu:~/git-conflict-practice$ git checkout main
Switched to branch 'main'
jf@ubuntu:~/git-conflict-practice$ git merge salty
Updating 813a2ee..ba47189
Fast-forward
 git-conflict-practice/recipe.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
jf@ubuntu:~/git-conflict-practice$ git merge sweet
Auto-merging git-conflict-practice/recipe.txt
CONFLICT (content): Merge conflict in git-conflict-practice/recipe.txt
Automatic merge failed; fix conflicts and then commit the result.
jf@ubuntu:~/git-conflict-practice$ cat recipe.txt
<<<<<<< HEAD
1 cup salt
=====
2 cups sugar
>>>>>>> sweet
2 cup water
jf@ubuntu:~/git-conflict-practice$ nano recipe.txt
jf@ubuntu:~/git-conflict-practice$ git add recipe.txt
jf@ubuntu:~/git-conflict-practice$ git commit
[main d23bed1] Merge branch 'sweet' into main
jf@ubuntu:~/git-conflict-practice$ cat recipe.txt
1 cup salt
2 cups sugar
```

图 31: 合并冲突的产生与解决过程



```
jf@ubuntu: ~/git-conflict-practice
../.gitignore_global
../.local/
../.mozilla/
../.pam_environment
../.presage/
../.profile
../.ssh/
../.sudo_as_admin_successful
../.xinputrc
../.xprofile
../.xsession-errors
../.xsession-errors.old
../Desktop/
../all.txt
../check.sh
../err.txt
../examples.desktop
../git-demo/
../notes.txt
../notes_2026_08-26.txt
../out.txt
../somefile

nothing added to commit but untracked files present (use "git add" to track)
jf@ubuntu:~/git-conflict-practice$ git log --graph --oneline
* d23bed1 Merge branch 'sweet' into main
|
| * a95e03a sweet: increase sugar quantity
| * | ba47189 salty: replace sugar with salt
|/
* 813a2ee init: basic recipe file
* 2ebe9e0 work of four q01-q04
jf@ubuntu:~/git-conflict-practice$
```

图 32: git log -graph -oneline 查看合并历史

3 解题感悟

3.1 遇到的问题

本次实验中遇到的问题主要有：

1. **含空格的文件名处理**：在 q01 复制带空格的 `notes one.txt` 时，若不使用引号，命令会被拆分为多个参数导致失败；
2. **合并冲突**：q03 与课后练习中，两个分支修改同一行导致冲突，需要手工判断保留内容并删除冲突标记；
3. **脚本无法直接运行**：写好的 `check.sh` 直接运行报 `Permission denied`，原因是缺少可执行权限；
4. **标准流混淆**：刚开始分不清 `stdout` 与 `stderr`，错误输出和正常输出混在一起。

3.2 解决过程

1. 对文件名加引号并使用 `find -exec sh -c` 批量处理，成功保留相对目录结构并处理了空格；
2. 通过查看 `<<<<<<< / ===== / >>>>>>>` 标记含义，理解了冲突来源，编辑保留 `mode=safe` 并添加 `note=reviewed` 完成合并；
3. 执行 `chmod +x` 并对比 `ls -l` 前后输出，理解了执行权限的意义；
4. 分别用 `>` 和 `2>` 重定向到不同文件，再用 `2>&1` 合并，彻底理解了三条标准流。

3.3 收获与反思

通过本次实验，我掌握了 Shell 文件系统操作、管道与文本处理、Git 分支与合并冲突的解决、以及 LaTeX 文档构建的基本技能。体会到：

- 命令行工具之间通过管道组合能高效完成任务，但要注意数据格式；
- Git 的冲突不是错误，而是需要人工决策的合并信号，理解冲突标记是解决冲突的关键；
- 养成使用 Git 分步提交、配置忽略文件的好习惯，能有效避免误提交和后期返工。

4 代码仓库与提交记录

按照课程要求，每次课程的报告、练习、代码等均上传到个人 GitHub 账号，保留 commit 记录，并禁止单次全量提交（分步、分主题多次提交）。

4.1 仓库链接

- GitHub 仓库地址：<https://github.com/GJF080117/jf>
- 本次实验对应分支：`main`

个人 GitHub 仓库 `jf` 主页截图如下，可看到按 `q01-q04`、`git-demo`、`git-conflict-practice` 等分目录组织的课程作业，以及各文件的提交记录：

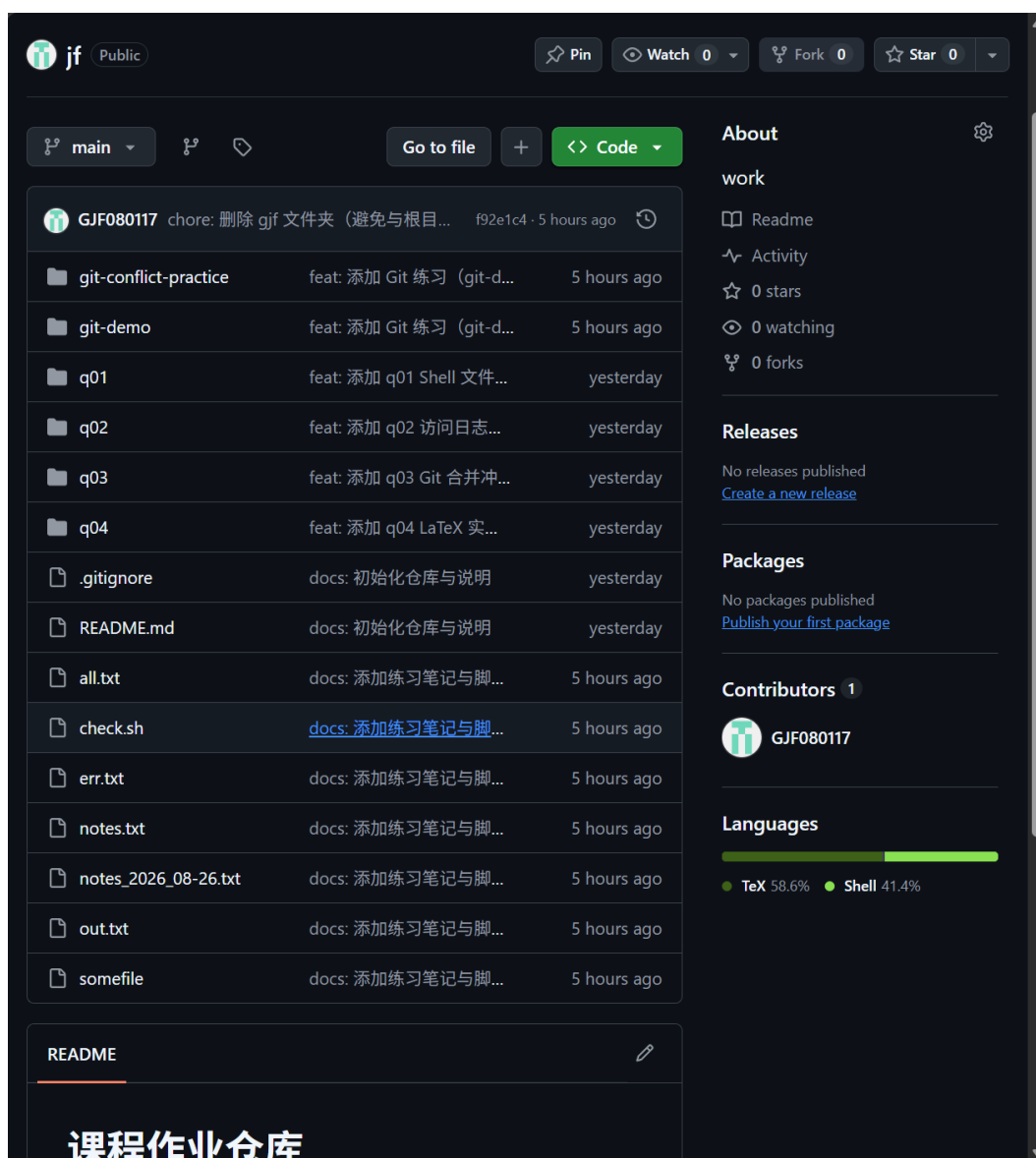


图 33: GitHub 仓库 jf 主页 (文件列表与提交记录)

4.2 commit 记录

本次实验的分步提交记录如下 (git log --oneline):

```

1 f92e1c4 chore: 删除 gjf 文件夹 (避免与根目录 q01-q04 重复)
2 e2e8b88 feat: 添加 gjf 作业文件夹
3 790ab3d docs: 添加练习笔记与脚本文件
4 6d9b53a feat: 添加 Git 练习 (git-demo 与 git-conflict-practice)
5 65e5d75 feat: 添加 q04 LaTeX 实验报告
6 6de435c feat: 添加 q03 Git 合并冲突实验
7 c96442f feat: 添加 q02 访问日志统计
8 223c32d feat: 添加 q01 Shell 文件系统实验
9 5a2a3e0 docs: 初始化仓库与说明
  
```